

OpenFOAM Programming

HybridFilmDetachmentInjection

Custom gravity + gas-shear surface film detachment model

Name	Satya Sai Phani Santosh Pilaka
Email	phainsantoshpss@gmail.com
OpenFOAM version	v2306

Project report and implementation notes

Abstract

HybridFilmDetachmentInjection is a custom runtime-selectable OpenFOAM surface-film injection model that combines two local detachment mechanisms: gravity-driven dripping and gas-shear stripping. The model evaluates a Bond-number criterion for gravity and a film Weber-number criterion for gas shear. Threshold exceedance is converted into a single detached film mass through a timestep-dependent rate law, while the droplet diameter is selected from a gravity correlation, a shear-stripping correlation, or a hybrid diameter when both mechanisms are active. The implementation is designed for integration with the single-layer surface-film framework and Lagrangian cloud coupling in OpenFOAM v2306. This document records the physical formulation, mathematical model, OpenFOAM architecture, current C++ implementation, planned verification cases, and literature basis.

Model-status note: The combined Bo/We trigger, exceedance weighting, exponential mass-removal law, and hybrid diameter interpolation are project-specific closures. The gravity diameter is based on the OpenFOAM BrunDrippingInjection/Lefebvre capillary-length treatment, while the shear diameter uses the empirical Sattelmayer prefiling-airblast correlation.

1. Project Objective and Scope

The objective is to develop a new runtime-selectable surface-film injection model named `hybridFilmDetachmentInjection`. The model removes liquid from an Eulerian surface film and transfers the detached mass to the Lagrangian cloud when either gravity or gas shear is sufficiently strong. The implementation is intended for cases based on the `thermoSingleLayer /kinematicSingleLayer` film infrastructure, including configurations derived from the `reactingParcelFoam` liquid-film tutorials.

- Gravity-driven detachment is represented with a Bond-number criterion.
- Gas-shear stripping is represented with a film Weber number criterion based on tangential gas-film relative velocity.
- Only one total detached mass is computed; separate drip and strip mass variables are deliberately not required.
- Mechanism exceedance is retained for choosing or blending the droplet-diameter closure.
- The model is registered in OpenFOAM at runtime through the `injectionModel` selection table.

2. Physical and Mathematical Formulation

2.1 Gravity-driven dripping

Gravity-driven detachment is activated only when the local gravity projection relative to the film surface is favorable for liquid to leave the wall. The model uses the film quantity `gNorm` together with local film and gas densities, film thickness, and surface tension.

$$Bo = ((\rho_l - \rho_g) g_n \delta^2) / \sigma$$

The Bond number compares gravitational forcing with capillary stabilization. Bond-number scaling is standard for free-surface systems, while the gravity-dripping motivation is consistent with Rayleigh–Taylor film instability on an inclined underside studied by Brun et al. [2]. OpenFOAM’s `BrunDrippingInjection` uses Brun et al. for the critical dripping behavior and Lefebvre for a capillary-length droplet diameter [1,3].

$$F_{Bo} = \max(Bo / Bo_{crit} - 1, 0)$$

`Bo_crit` is a user-defined threshold in the present hybrid model. `F_Bo` is therefore a dimensionless exceedance measure rather than a published Brun-model parameter.

2.2 Gas-shear stripping

Aerodynamic stripping is based on the gas-film relative velocity parallel to the wall. The normal component is removed so that the stripping criterion represents tangential gas action on the film.

$$U_{rel} = U_g - U_f$$

$$U_{rel,t} = U_{rel} - (U_{rel} \cdot n) n$$

$$We_f = \rho_g |U_{rel,t}|^2 \delta / \sigma$$

$$F_{We} = \max(We_f / We_{crit} - 1, 0)$$

The Weber number compares aerodynamic/inertial forcing with surface-tension resistance. We_{crit} is a calibration parameter of this project rather than a universal stripping threshold. The use of a Weber-number control is physically consistent with atomization and film-breakup literature [4,5].

2.3 Combined exceedance and detached mass

$$E = F_{Bo} + F_{We}$$

$$\tau_\sigma = (\rho_l \delta^3 / \sigma)^{0.5}$$

$$\lambda = C_m E / \tau_\sigma$$

$$f_{det} = 1 - \exp(-\lambda \Delta t)$$

$$m_{leave} = f_{det} m_{available}$$

The capillary-inertial timescale provides a local response scale. The exponential removal law is a project-specific closure selected so that the detached fraction tends to zero continuously at the threshold and remains timestep-aware. It should therefore be validated through timestep-sensitivity tests. The model intentionally computes one total detached mass rather than separate dripping and stripping masses.

2.4 Mass conservation

$$massToInject \leftarrow massToInject + m_{leave}$$

$$availableMass \leftarrow availableMass - m_{leave}$$

The same mass is added to the cloud-transfer accumulator and subtracted from the film's available mass. The cumulative injected-mass bookkeeping is updated through `addToInjectedMass(massLeave)`, following the surface-film injection-model pattern used by OpenFOAM [1].

3. Droplet-Diameter Models

3.1 Gravity-only diameter

$$l_c = (\sigma / (\rho_l |g|))^{0.5}$$

$$d_{Bo} = C_d l_c, \text{ with } C_d = 3.3 \text{ by default}$$

This is the capillary-length relation used in OpenFOAM's `BrunDrippingInjection`. The OpenFOAM documentation states that the droplet diameter is obtained from the local capillary length multiplied by `dCoeff`, with a default value of 3.3, and cites Lefebvre's *Atomization and Sprays* [1,3].

3.2 Shear-only diameter

$$d_{We} = 0.67 \sigma^{0.75} / U_g^{1.57}$$

The shear-stripping diameter is based on the empirical Sattelmayer correlation for prefilming airblast atomization. Sattelmayer and Wittig showed that droplet size is strongly governed by air velocity and surface tension [4]. Later literature reproduces Sattelmayer's correlation in the form above and reports original flat-plate air velocities of approximately 30–120 m/s [6]. The coefficient 0.67 is dimensional for the units used by the correlation; it is not a universal dimensionless constant.

Applicability warning: The Sattelmayer expression was developed for a planar prefilming-airblast atomiser, not for arbitrary wall-film stripping. In this project, it is used as a physically motivated empirical closure and must be validated before quantitative claims are made, especially outside the original velocity/geometry range.

3.3 Hybrid diameter when both mechanisms are active

$$W_{Bo} = F_{Bo} / (F_{Bo} + F_{We})$$

$$W_{We} = F_{We} / (F_{Bo} + F_{We})$$

$$d_{hybrid} = 1 / (W_{Bo}/d_{Bo} + W_{We}/d_{We})$$

This weighted harmonic interpolation is a project-specific hybrid closure. It is motivated by the volume-to-surface-area structure of the Sauter mean diameter, but F_{Bo} and F_{We} are exceedance weights, not independently measured mass fractions. The report should therefore identify this expression as the present model's proposed blending rule, not as a published Sattelmayer or Lefebvre formula.

4. OpenFOAM Architecture

The class derives from `surfaceFilmModels::injectionModel`. OpenFOAM v2306 provides runtime-selectable surface-film submodels in the `Foam::regionModels::surfaceFilmModels` namespace [7]. The `kinematicSingleLayer` class exposes film fields such as `U()`, `delta()`, `rho()`, `sigma()`, `nHat()`, `gNorm()`, as well as mapped primary-region properties such as `UPrimary()` and `rhoPrimary()` [8].

4.1 Runtime selection

Code excerpt: runtime-selection registration

```
defineTypeNameAndDebug(hybridFilmDetachmentInjection, 0);
addToRunTimeSelectionTable
(
    injectionModel,
    hybridFilmDetachmentInjection,
    dictionary
);
```

The `TypeName` entry in the header and `addToRunTimeSelectionTable` entry in the source make the model selectable from `surfaceFilmProperties` without editing the solver source.

4.2 Film and gas fields

Code excerpt: local film and mapped primary-phase properties

```
const scalarField& delta = film.delta();
const scalarField& rho = film.rho();
const scalarField& sigma = film.sigma();
const volVectorField& Ufilm = film.U();

const volVectorField& Ugas = film.UPrimary();
const scalarField& rhoGas = film.rhoPrimary();
```

`UPrimary` and `rhoPrimary` are fields mapped from the primary region to the film region in the single-layer film framework [8]. This makes the local gas-film comparison possible inside the injection submodel.

5. Implementation Details

5.1 Header file (.H)

The header declares the custom model, its runtime type information, constructor/destructor, the correct() interface, and the model coefficients. The present implementation uses four principal coefficients: bondNumCrit_, weberNumCrit_, cm_, and dCoeff_.

5.2 Constructor and dictionary coefficients

Code excerpt: constructor

```
hybridFilmDetachmentInjection::hybridFilmDetachmentInjection
(
    surfaceFilmRegionModel& film,
    const dictionary& dict
)
:
    injectionModel(type(), film, dict),
    bondNumCrit_ (coeffDict_.getOrDefault<scalar>("bondNumCrit", 1)),
    weberNumCrit_ (coeffDict_.getOrDefault<scalar>("weberNumCrit", 5)),
    cm_ (coeffDict_.getOrDefault<scalar>("cm", 0.25)),
    dCoeff_ (coeffDict_.getOrDefault<scalar>("dCoeff", 3.3))
{}

```

Parameter	Meaning	Default	Status
bondNumCrit	Critical Bond number	1.0	Project parameter
weberNumCrit	Critical film Weber number	5.0	Project parameter
cm	Detachment-rate coefficient	0.25	Project parameter / calibration
dCoeff	Gravity diameter coefficient	3.3	OpenFOAM/Lefebvre basis [1,3]

5.3 correct() workflow

- Obtain the kinematicSingleLayer reference and film/primary fields.
- For each film cell, compute tangential gas-film relative velocity.
- Compute Bond number only for a favorable gravity orientation.
- Compute the local film Weber number.
- Calculate F_{Bo} and F_{We} and combine them as E .
- Use the capillary timescale and exponential rate law to obtain f_{det} and massLeave.
- If mass leaves the film, update massToInject, availableMass, and cumulative injected mass.
- Select d_{Bo} , d_{We} , or the hybrid diameter according to the active mechanism(s).
- Call injectionModel::correct() after processing the cells.

Code excerpt: exceedance-to-mass calculation

```
const scalar FBo = max(bondNum/bondNumCrit_ - 1.0, 0.0);
const scalar FWe = max(weberNum/weberNumCrit_ - 1.0, 0.0);
const scalar E = FBo + FWe;

scalar tau = max
(
    pow((rho[cellI]*pow(delta[cellI], 3))/sigma[cellI], 0.5),
    SMALL
);

scalar lambda = (cm_*E)/tau;
scalar fDet = min(max(1 - exp(-lambda*deltaT), 0.0), 1.0);
scalar massLeave = fDet*availableMass[cellI];

```

Code excerpt: diameter selection

```
if (FBo > 0 && FWe == 0)
{

```

```

    diameterToInject[cellI] = dBo;
}
else if (FBo == 0 && FWe > 0)
{
    diameterToInject[cellI] = dWe;
}
else if (FBo > 0 && FWe > 0)
{
    const scalar Wbo = FBo/(FBo + FWe);
    const scalar Wwe = FWe/(FBo + FWe);

    diameterToInject[cellI] =
        1.0/((Wbo/dBo) + (Wwe/dWe));
}

```

6. Runtime Dictionary Integration

The custom film injection model is selected in `surfaceFilmProperties`. The coefficient names must match the constructor keywords exactly.

Code excerpt: `surfaceFilmProperties`

```

injectionModels
(
    hybridFilmDetachmentInjection
);

hybridFilmDetachmentInjectionCoeffs
{
    bondNumCrit    1.0;
    weberNumCrit   5.0;
    cm             0.25;
    dCoeff         3.3;
}

```

7. Custom Library Structure and Compilation

The intended custom-library layout is:

```

HybridFilmDetachmentInjection/
├── hybridFilmDetachmentInjection.H
├── hybridFilmDetachmentInjection.C
├── Make/
│   ├── files
│   └── options

```

8. Verification and Test Matrix

The first verification should isolate each branch of the model before using a full sprinkler case. Suggested tests are qualitative branch tests rather than experimental validation of the model coefficients.

Case	Film thickness δ	Relative velocities	Expected branch	Checks
A	1×10^{-3} m	2 m/s	No detachment	$F_{Bo}=0, F_{We}=0,$ $massLeave=0$
B	3×10^{-3} m	2 m/s	Gravity only	$F_{Bo}>0, F_{We}=0,$ $d=d_{Bo}$
C	1×10^{-3} m	20 m/s	Shear only	$F_{Bo}=0, F_{We}>0,$ $d=d_{We}$
D	3×10^{-3} m	10 m/s	Hybrid	$F_{Bo}>0, F_{We}>0,$ $d=d_{hybrid}$

For numerical verification, write or print local values of Bo , We_f , F_{Bo} , F_{We} , E , τ , f_{det} , $massLeave$, and $diameterToInject$ at selected cells. Verify that `availableMass` decreases by the same amount that `massToInject` increases.

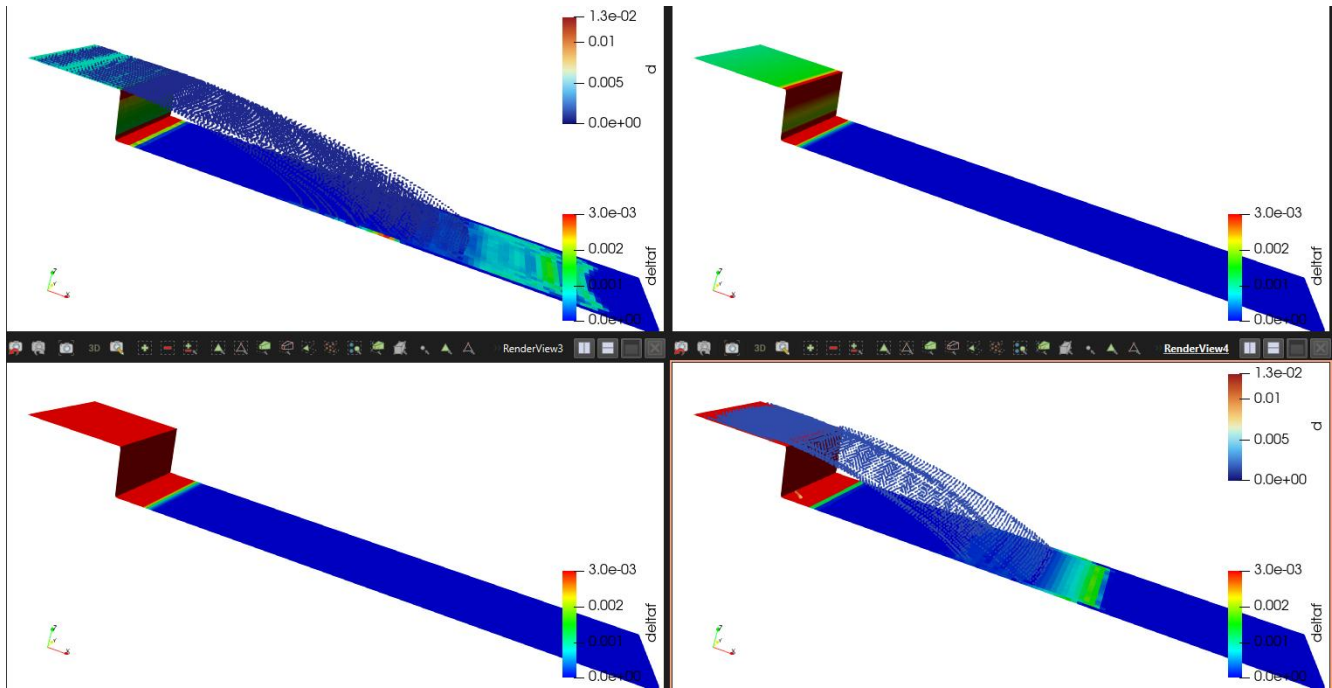


Fig. 1: Contours of all four test cases at time step 1second.

Top left: Case C, Top Right: Case B, Bottom Left: Case A, Bottom Right: Case D

11. Assumptions and Limitations

- $Bo_{crit} = 1$ and $We_{crit} = 5$ are model inputs and require sensitivity analysis or calibration; they should not be presented as universal critical values.
- The exponential detachment fraction and coefficient C_m are proposed project closures and require timestep and parameter sensitivity studies.
- The Sattelmayer diameter correlation was obtained for prefilming-airblast atomization and is applied here outside its original geometry as an engineering closure.
- The published shear diameter relation uses gas velocity, whereas the detachment trigger uses tangential gas-film relative velocity. This distinction should be retained explicitly in the report.
- When both mechanisms are active, the harmonic blend is an SMD-inspired interpolation using exceedance weights; it is not a literature correlation.
- Secondary breakup, atomization, evaporation, and collision models can alter the parcel population after film injection and should be disabled during initial model verification.
- The main assumption includes consideration of the laminar regime to maintain the model simplicity even though the gas speeds fall into the turbulent domain, which might include certain physical errors.

12. Conclusion

The hybridFilmDetachmentInjection project extends the OpenFOAM surface-film injection architecture with a transparent two-trigger formulation: a gravity Bond number and a gas-shear film Weber number. The present design retains a single detached mass for simplicity and conservation, while mechanism information is used only to select the droplet-diameter closure.

Appendix A. Current Header Implementation (.H)

```

/*-----*\
=====
\ \ / F i e l d   | OpenFOAM: The Open Source CFD Toolbox
\ \ / O p e r a t i o n |
\ \ / A n d       | www.openfoam.com
\ \ M a n i p u l a t i o n |
-----

```

Copyright (C) 2016-2017 OpenFOAM Foundation

License

This file is part of OpenFOAM.

OpenFOAM is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

OpenFOAM is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with OpenFOAM. If not, see <<http://www.gnu.org/licenses/>>.

Class

Foam::regionModels::surfaceFilmModels::hybridFilmDetachmentInjection

Description

Hybrid surface-film detachment and cloud-injection model combining gravity-driven dripping and aerodynamic gas-shear stripping.

A local Bond number characterises gravity detachment

$$Bo = (\rho_l - \rho_g) * g_n * \delta^2 / \sigma$$

and shear stripping by a film Weber number based on the tangential gas-film relative velocity

$$We_f = \rho_g * |U_g - U_{f,t}|^2 * \delta / \sigma.$$

Detachment is activated when either dimensionless number exceeds its user-specified critical value. The amount of film mass transferred to the Lagrangian cloud is determined from the combined threshold exceedance using a capillary-timescale-based rate law.

For gravity-only detachment, the injected droplet diameter follows the capillary-length relation used by OpenFOAM BrunDrippingInjection, $d = dCoeff * \sqrt{\sigma / (\rho_l * g)}$, with $dCoeff = 3.3$ by default.

For shear-only detachment, the Sauter mean diameter is estimated using the empirical Sattelmayer prefilming-airblast correlation $d_{32} = 0.67 * \sigma^{0.75} / U_g^{1.57}$.

If both mechanisms are active, the present model uses an SMD-inspired weighted harmonic interpolation between the gravity and shear diameter estimates. The weighting and mass-detachment rate law are model-specific closures introduced in this project.

References

Brun, P. T., Damiano, A., Rieu, P., Balestra, G., & Gallaire, F. (2015). Rayleigh-Taylor instability under an inclined plane. *Physics of Fluids*, 27(8), 084107. doi:10.1063/1.4927857

Lefebvre, A. H. (1988). *Atomization and Sprays*. CRC Press.

Sattelmayer, T. (1985). Zum Einfluss der ausgebildeten, turbulenten Luft-Fluessigkeitsfilm-Stromung auf den Filmzerfall und die Tropfenbildung am Austritt von Spalten geringer Hoehe. Dissertation, Universitaet Karlsruhe.

Sattelmayer, T., & Wittig, S. (1986). Internal Flow Effects in Prefilming Airblast Atomizers: Mechanisms of Atomization and Droplet Spectra. *Journal of Engineering for Gas Turbines and Power*, 108(3), 465-472. doi:10.1115/1.3239931

SourceFiles

hybridFilmDetachmentInjection.C

```

\*-----*/
#ifndef hybridFilmDetachmentInjection_H
#define hybridFilmDetachmentInjection_H

#include "injectionModel.H"

namespace Foam
{
namespace regionModels
{
namespace surfaceFilmModels
{

class hybridFilmDetachmentInjection
:
public injectionModel
{
// Private member functions

//- No copy construct
hybridFilmDetachmentInjection(const hybridFilmDetachmentInjection&) = delete;

//- No copy assignment
void operator=(const hybridFilmDetachmentInjection&) = delete;

protected:

// Protected data

//- Critical Bond number, default = 1
scalar bondNumCrit ;

//- Critical Weber num, default = 5
scalar weberNumCrit ;

//- Detachment rate coefficient, default = 0.25
scalar cm ;

//- Coefficient for droplet diameter, default = 3.3
scalar dCoeff ;

//- Diameters of particles to inject into the dripping
scalarList diameter ;

public:

//- Runtime type information
TypeName("hybridFilmDetachmentInjection");

// Constructors

//- Construct from surface film model
hybridFilmDetachmentInjection
(
surfaceFilmRegionModel& film,
const dictionary& dict
);

//- Destructor
virtual ~hybridFilmDetachmentInjection();

// Member Functions

//- Correct
virtual void correct
(
scalarField& availableMass,
scalarField& massToInject,
scalarField& diameterToInject
);
};

} // End namespace surfaceFilmModels
} // End namespace regionModels
} // End namespace Foam

#endif

```

Appendix B. Current Source Implementation (.C)

```

/*-----*\
=====
\\ / F i e l d   | OpenFOAM: The Open Source CFD Toolbox
\\ / O p e r a t i o n   |
\\ / A n d       | www.openfoam.com
\\ M a n i p u l a t i o n   |
-----*/

Copyright (C) 2016-2017 OpenFOAM Foundation
Copyright (C) 2020 OpenCFD Ltd.
-----*/

License
This file is part of OpenFOAM.

OpenFOAM is free software: you can redistribute it and/or modify it
under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
for more details.

You should have received a copy of the GNU General Public License
along with OpenFOAM. If not, see <http://www.gnu.org/licenses/>.

\*-----*/

#include "hybridFilmDetachmentInjection.H"
#include "addToRunTimeSelectionTable.H"
#include "kinematicSingleLayer.H"

namespace Foam
{
    namespace regionModels
    {
        namespace surfaceFilmModels
        {
            defineTypeNameAndDebug(hybridFilmDetachmentInjection, 0);
            addToRunTimeSelectionTable(injectionModel, hybridFilmDetachmentInjection, dictionary);

            hybridFilmDetachmentInjection::hybridFilmDetachmentInjection
            (
                surfaceFilmRegionModel& film,
                const dictionary& dict
            )
            :
                injectionModel(type(), film, dict),
                bondNumCrit (coeffDict_.getOrDefault<scalar>("bondNumCrit", 1)),
                weberNumCrit (coeffDict_.getOrDefault<scalar>("weberNumCrit", 5)),
                cm_(coeffDict_.getOrDefault<scalar>("cm", 0.25)),
                dCoeff_(coeffDict_.getOrDefault<scalar>("dCoeff", 3.3))
            {}

            hybridFilmDetachmentInjection::~hybridFilmDetachmentInjection()
            {}

            void hybridFilmDetachmentInjection::correct
            (
                scalarField& availableMass,
                scalarField& massToInject,
                scalarField& diameterToInject
            )
            {
                const kinematicSingleLayer& film =
                    refCast<const kinematicSingleLayer>(this->film());

                tmp<volScalarField> tgNorm(film.gNorm());
                const scalarField& gNorm = tgNorm();
                const scalar magg = mag(film.g().value());
                const volVectorField& nHat = film.nHat();

```

```

const scalarField& delta = film.delta();
const scalarField& rho = film.rho();
const scalarField& sigma = film.sigma();
const volVectorField& UFilm = film.U();

const volVectorField& Ugas = film.UPrimary();
const scalarField& rhoGas = film.rhoPrimary();

const fvMesh& mesh = film.regionMesh();
const scalar deltaT = mesh.time().deltaTValue();

forAll(delta, cellI)
{
    bool dripOrStrip = false;
    const vector UrelVec = Ugas[cellI] - UFilm[cellI];
    const vector UrelTan = UrelVec - (UrelVec & nHat[cellI])*nHat[cellI];
    const scalar Urel = mag(UrelTan);
    const scalar magUg = mag(Ugas[cellI]);

    scalar bondNum = 0;

    if(gNorm[cellI] > SMALL)
    {
        bondNum = ((rho[cellI] - rhoGas[cellI]) * gNorm[cellI]
            * pow(delta[cellI],2)) / sigma[cellI];
    }

    scalar weberNum = (rhoGas[cellI] * pow(Urel, 2)
        * delta[cellI])/sigma[cellI];

    const scalar FBo = max(bondNum/bondNumCrit_ - 1.0, 0.0);
    const scalar FWe = max(weberNum/weberNumCrit_ - 1.0, 0.0);

    const scalar E = FBo + FWe;

    scalar tau = max(pow(((rho[cellI] * pow(delta[cellI], 3))
        /sigma[cellI]), 0.5), SMALL);

    scalar lambda = (cm_ * E) / tau;

    scalar fDet = min(max((1 - (exp(-lambda * deltaT))), 0.0), 1.0);

    scalar massLeave = fDet * availableMass[cellI];

    if(massLeave > 0)
    {
        massToInject[cellI] += massLeave;
        availableMass[cellI] -= massLeave;
        addToInjectedMass(massLeave);

        scalar lc = sqrt(sigma[cellI]/(rho[cellI] * magg));
        scalar dBo = dCoeff * lc;
        scalar dWe = 0.67 * ((pow(sigma[cellI], 0.75))
            / (pow(magUg, 1.57)));

        if(FBo > 0 && FWe == 0)
        {
            scalar diam = dBo;
            diameterToInject[cellI] = diam;
        }
        else if(FBo == 0 && FWe > 0)
        {
            scalar diam = dWe;
            diameterToInject[cellI] = diam;
        }
        else if(FBo > 0 && FWe > 0)
        {
            const scalar Wbo = (FBo/(FBo + FWe));
            const scalar Wwe = (FWe/(FBo + FWe));

            scalar diam = 1/((Wbo / dBo) + (Wwe / dWe));
            diameterToInject[cellI] = diam;
        }

        dripOrStrip = true;
    }

    if(!dripOrStrip)
    {
        diameterToInject[cellI] = 0;
        massToInject[cellI] = 0;
    }
}

injectionModel::correct();
}

```

```

} // End namespace surfaceFilmModels
} // End namespace regionModels
} // End namespace Foam

```

References

- [1] OpenCFD Ltd., OpenFOAM API Guide, BrunDrippingInjection class and source documentation. The model documentation states that the droplet diameter is the local capillary length multiplied by dCoeff (default 3.3) and cites Brun et al. and Lefebvre. <https://api.openfoam.com/>
- [2] P. T. Brun, A. Damiano, P. Rieu, G. Balestra, and F. Gallaire, “Rayleigh–Taylor instability under an inclined plane,” *Physics of Fluids*, vol. 27, no. 8, 084107, 2015. doi: 10.1063/1.4927857.
- [3] A. H. Lefebvre, *Atomization and Sprays*. CRC Press, 1988. ISBN 9780891166030.
- [4] T. Sattelmayer and S. Wittig, “Internal Flow Effects in Prefilming Airblast Atomizers: Mechanisms of Atomization and Droplet Spectra,” *Journal of Engineering for Gas Turbines and Power*, vol. 108, no. 3, pp. 465–472, 1986. doi: 10.1115/1.3239931.
- [5] A. H. Lefebvre and V. G. McDonell, *Atomization and Sprays*, 2nd ed. CRC Press, 2017. ISBN 9781498736251.
- [6] Safiullah, B. Esquivias, B. Hickey, and V. McDonell, “Investigation of Liquid Film Behavior on the Surface of Airfoil and Subsequent Atomization from Trailing Edge – Effect of Liquid Physical Properties,” ILASS-Americas 34th Annual Conference on Liquid Atomization and Spray Systems, 2024. The paper reproduces Sattelmayer’s $SMD = 0.67 \sigma^{0.75} / u_g^{1.57}$ relation and reports the original Sattelmayer/Wittig flat-plate air-velocity range of approximately 30–120 m/s.
- [7] OpenCFD Ltd., OpenFOAM v2306 API Guide, Foam::regionModels::surfaceFilmModels namespace. https://api.openfoam.com/2306/namespaceFoam_1_1regionModels_1_1surfaceFilmModels.html
- [8] OpenCFD Ltd., OpenFOAM API Guide, kinematicSingleLayer class/source: UPrimary and rhoPrimary are mapped primary-region properties available in the film region. <https://api.openfoam.com/>